

PROJECT REPORT

Biologically Inspired Artificial Intelligence Lunar Lander - Perfect Landing Neuroevolution

- Karol Biezuński github.com/delux444
- Michał Figołuszka github.com/Michaleq24

Contents:

Authors	1
1. Introduction.....	2
2. Analysis of the task	2
a. Possible approaches to solve the problem	2
b. Presentation of available datasets.....	3
c. Analysis of available tools/frameworks	3
3. Internal and external specification of the software solution	4
a. Classes / Methods / Functions	4
b. Data structures	4
c. User interface	5
4. Presentation of the Results.....	5
5. Summary & conclusions.....	7
6. References	7
7. Repository Link.....	8

1. Introduction

The objective of this project is to develop an autonomous flight pilot capable of successfully landing a spacecraft in the OpenAI Gymnasium LunarLander-v3 environment. Unlike standard deep reinforcement learning approaches (such as DQN or PPO) that rely on gradient descent and backpropagation, this project utilizes **Neuroevolution**. By employing a Genetic Algorithm, the neural network's weights are evolved directly over multiple generations based on the principles of natural selection and survival of the fittest. The ultimate mission is to achieve a "Perfect Landing" — a smooth, precise touchdown between two flags without crashing.

2. Analysis of the task

a. Possible approaches to solve the problem

- **Deep Reinforcement Learning (e.g., DQN, PPO):**
 - *Pros:* Mathematically grounded, heavily supported by modern libraries (Stable-Baselines3), capable of solving complex continuous/discrete control tasks efficiently.
 - *Cons:* Requires complex gradient calculations, backpropagation, and extensive hyperparameter tuning. Prone to issues like vanishing gradients.

- **Neuroevolution (Genetic Algorithms - GA):**
 - *Pros:* Gradient-free optimization, meaning it does not suffer from vanishing gradients. Highly intuitive, naturally supports exploration through mutation, and trivially parallelizable.
 - *Cons:* Can be slower to converge in environments with massive state spaces; relies heavily on population size and randomly generated mutations.
- **Selected methodology:** Neuroevolution was chosen for this project. A feedforward neural network acts as the agent's policy, but instead of updating weights via backpropagation, the entire weight and bias matrix is flattened into a 1D vector (chromosome). A Genetic Algorithm evaluates, selects, breeds, and mutates a population of these networks to maximize a custom fitness score.

b. Presentation of available datasets

In reinforcement learning and neuroevolution tasks, static datasets are not used. Instead, data is generated dynamically through agent-environment interaction.

- **The Dataset (Environment State):**
The LunarLander-v3 environment provides an 8-dimensional continuous observation vector for every step:
 1. x - horizontal position (0 = pad center)
 2. y - vertical position (0 = ground)
 3. vx - horizontal velocity
 4. vy - vertical velocity
 5. angle - spacecraft body angle (0 = upright)
 6. angular_velocity
 7. leg_left_contact (Y/N)
 8. leg_right_contact (Y/N)

c. Analysis of available tools/frameworks

- **Gymnasium (by Farama Foundation):** Chosen for the environment simulation. It is the modern standard for RL tasks, providing the LunarLander-v3 physics simulation built on Box2D.
- **PyGAD:** An open-source Python library for building genetic algorithms and optimizing machine learning algorithms. Chosen because it provides robust, ready-to-use GA operations (crossover, mutation, elitism) without the need to write them from scratch.
- **NumPy:** Selected for fast, efficient matrix operations during the forward pass of the neural network.

3. Internal and external specification of the software solution

a. Classes / Methods / Functions

The project is structurally contained in a single Python script (main.py) utilizing functional paradigms for speed and simplicity.

- `unpack(w)`: Slices the 1D chromosome flat array back into structured weight matrices (w_1, w_2, w_3) and bias vectors (b_1, b_2, b_3) for the neural network layers.
- `forward(obs, w)`: Executes the neural network forward pass. Uses $\tanh$ activation functions for hidden layers and an argmax function at the output layer to select the discrete action (0: Do nothing, 1: Fire left engine, 2: Fire main, 3: Fire right).
- `get_env()`: A singleton-like function managing the Gymnasium environment instance. It ensures isolated access, avoiding Box2D threading crashes.
- `fitness_func(ga_instance, solution, solution_idx)`: The core evaluation loop. Runs the neural network (solution) through 5 episodes in the environment. It calculates custom rewards based on precision, soft landing, and upright angles.
- `on_generation(ga_instance)`: A callback triggered after every generation. It handles dynamic dashboard rendering, stagnation detection, and temporary mutation boosting.
- `run_test_flight(weights, render=True)`: Loads the best weights after training and renders the environment for visual verification.

b. Data structures

- **Chromosome (Genome)**: A 1D floating-point NumPy array representing all weights and biases. In the final architecture ($8 \rightarrow 64 \rightarrow 32 \rightarrow 4$), the chromosome length is 2,788 genes.
- **Action Space**: Integer values from 0 to 3.
- **Network Architecture**: Fully connected feedforward network with two hidden layers.
 - **Input** [8] $\rightarrow$ **Hidden 1** [64, $\tanh$] $\rightarrow$ **Hidden 2** [32, $\tanh$] $\rightarrow$ **Output** [4, argmax].

c. User interface

- **Console Dashboard:** During training, the software features a live, in-place updating ANSI console dashboard. It displays the progress bar, current generation, best fitness score, stagnation indicator, elapsed time, and ETA.
- **Graphical Visualizer:** Upon training completion, the `run_test_flight` function triggers a PyGame-based window (provided by Gym's `render_mode="human"`) to visually display the spacecraft's landing behavior in real-time.

4. Presentation of the results

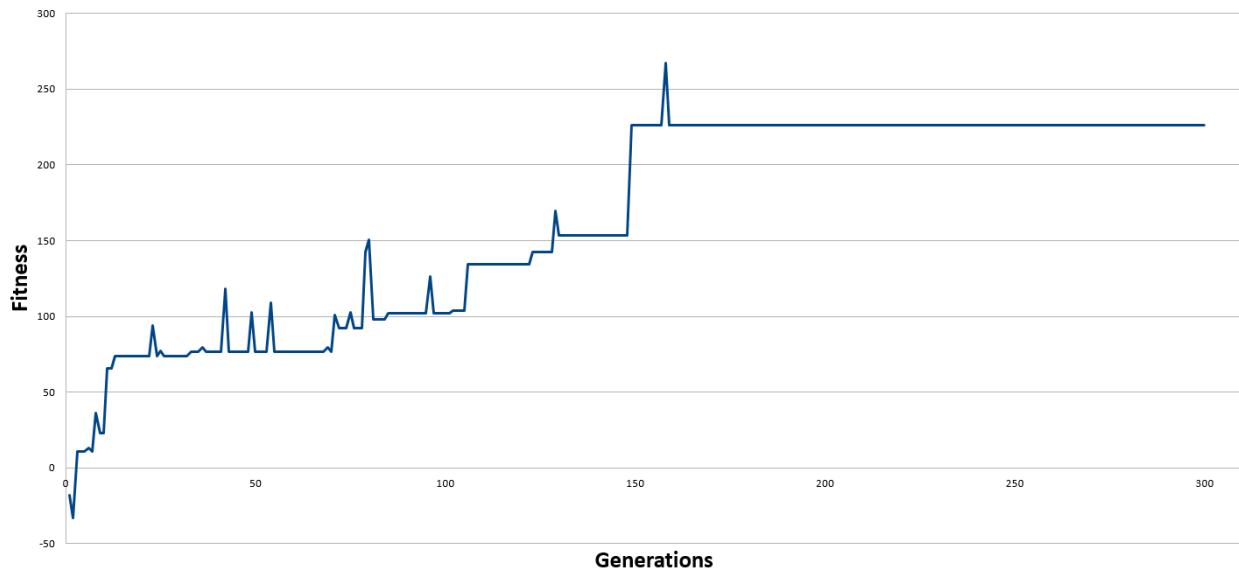
Experiment: Deep Network & Precision Reward Shaping

- *Parameters:* To solve the complexity of a stable landing, network size was increased to $8 \rightarrow 64 \rightarrow 32 \rightarrow 4$ (2788 genes). Population increased to 100, Generations to 300. Crossover changed to "two_points" to preserve weight matrix structures. Adaptive mutation was used: high (0.25) for weak agents, low (0.03) for elites.
- *Reward shaping:* Added three new penalties/bonuses calculated upon landing:
 1. *Precision bonus:* Up to +80 for landing close to $x=0$. Calculated as $1.0 - \text{abs}(x_error) / 0.3$.
 2. *Soft landing bonus:* Up to +40 for low vertical speed. Calculated as $1.0 - \text{abs}(v_y) / 1.5$.
 3. *Upright bonus:* Up to +30 for small angle at touchdown.

```
=====
LUNAR LANDER - PRECISION GA TRAINER
=====
Network  : 8 → 64 → 32 → 4
Genes    : 2788
Pop size : 100   Generations: 300
Episodes : 5 per fitness call
=====

  LUNAR LANDER - PRECISION TRAINER
  [=====] 2.3% ||
  Generation : 7 / 300
  Best Fit    : 70.93 Learning...
  Stagnation  : [=====] 1/25 ||
  Elapsed    : 27.7s avg 3.95s/gen
  ETA        : 1157.9s
  Press Ctrl+C to stop early and test best agent
```

P1. Screenshot of learning process degree.



P5. Data chart for top fitness points over number of generations.

- **Analysis of Results:** Large population size (100 for the tests) and increased genetic diversity allowed the algorithm to escape local optima. The implementation of stagnation detection (temporarily raising mutation rate by 1.4x if the best fitness didn't improve for 25 generations) prevented premature convergence. The change in reward shaping forced the neural network to prioritize slow, vertical descents strictly inside the flag area. The fitness score reliably surpassed the 200-point threshold (considered "solved" by Gymnasium standards) around generation 100-150.

5. Summary & conclusions

The project successfully demonstrated that a completely gradient-free Genetic Algorithm can solve complex continuous-observation physics simulations like LunarLander-v3. We observed that Neuroevolution is highly sensitive to the fitness function design. As proven in early experiments, minor loopholes in reward shaping will be aggressively exploited by the Genetic Algorithm. A structured, multi-layer fitness function combined with adaptive mutation and elitism was the key to achieving a stable, perfect landing.

6. References

1. Farama Foundation, *Gymnasium Documentation - LunarLander-v3*. Available at: https://gymnasium.farama.org/environments/box2d/lunar_lander/
2. Gad, A. F., *PyGAD: An Intuitive Genetic Algorithm Python Library*. Available at: <https://pygad.readthedocs.io/>
3. NumPy Developers, *NumPy Documentation*. Available at: <https://numpy.org/doc/>
4. OpenAI, *Box2D Physics Engine documentation*.

7. Repository Link

Project Files Location: <https://github.com/delux444/BIAl-project-gad-lunar-lander>